Dokumentation einer WGS-Variantenpipeline mit Snakemake: von FASTQ-Rohdaten zu annotierten ClinVar-Kandidaten und einem Elternvergleich (NA12878, NA12877)

Karim Andjar

Justus-Liebig-Universität Gießen, Master Bioinformatik und Systembiologie

13. Oktober 2026

Zusammenfassung

Diese Dokumentation beschreibt eine vollständige Pipeline zur Auswertung menschlicher Ganzgenomsequenzierungen (WGS) auf einem Laptop. Ausgehend von den Rohdaten zweier Proben des Platinum-Genomes-Pedigrees, NA12878 (Mutter, Lauf ERR194147) und NA12877 (Vater, Lauf ERR194146), jeweils etwa 50-fache Tiefe, werden Qualitätskontrolle, Trimmen, Mapping, Duplikatmarkierung, Variantenbestimmung, Annotation mit VEP und ClinVar, die Filterung pathogener Kandidaten und eine Qualitätsprüfung anhand der Allelbalance als Snakemake-Workflow und ergänzende Python-Skripte umgesetzt. Ein Vergleichsskript stellt die Kandidaten beider Eltern gegenüber. Für jeden Schritt werden die biologische Idee, das verwendete Werkzeug, die Snakemake-Regel, die Ein- und Ausgabedateien und die tatsächlich erzielten Zahlen erklärt. Ergänzt wird dies durch eine Schritt-für-Schritt-Anleitung zur Reproduktion, eine Liste der aufgetretenen Probleme mit ihren Lösungen und eine offene Darstellung der Grenzen, insbesondere der noch fehlenden Validierung gegen ein Truth Set.

Schlüsselwörter: Snakemake, WGS, BWA-MEM, GATK, VEP, ClinVar, Reproduzierbarkeit

Tabelle 1: Verwendete Werkzeuge und Versionen.

Werkzeug	Version	Aufgabe
FastQC	0.12.1	Qualitätsbericht
fastp	1.3.7	Trimmen, Filtern
BWA (MEM)	0.7.19	Mapping
samtools	1.24	Sortieren, Index, Statistik
GATK	4.6.2.0	Duplikate, Variantenbestimmung
Snakemake	6.1.2	Workflow
Ensembl VEP	116.2	Annotation
ClinVar	2026-10-04	Pathogenitätsangaben
Python	3.10	Auswertung, Abbildungen
pycirclize	1.10.1	Circos-Plot

1 Überblick

Die Pipeline beantwortet eine Frage: Welche Varianten eines Genoms sind in ClinVar als pathogen eingetragen, wie belastbar sind diese Treffer technisch, und welche Gene tragen beide Eltern eines Paares? Die Pipeline läuft für jede Probe getrennt mit identischen Schritten; erst der Elternvergleich führt beide Ergebnisse zusammen. Der Datenfluss ist in Abbildung 1 dargestellt. Die blauen Schritte erzeugen aus den Reads eine Variantenliste, die orangen Schritte annotieren, filtern und prüfen sie.

Tabelle 2: Größe der wichtigsten Dateien (Mutter; beim Vater gleiche Größenordnung).

Datei	Größe
Rohdaten (2 FASTQ-Dateien)	48 + 49 GB
Getrimmte FASTQ-Dateien	je 48 GB
`sorted.bam`	96 GB
`dedup.bam`	113 GB
Referenz mit BWA-Index	etwa 5,8 GB
VEP-Cache (Archiv)	25,7 GB
ClinVar	191 MB

2 Umgebung und Installation

2.1 Rechner und Speicherbedarf

Alles lief auf einem MacBook Air (Apple Silicon, 25 GB Arbeitsspeicher). Der Speicherbedarf ist der eigentliche Engpass (Tab. 2): Im Spitzenwert liegen alle Zwischendateien einer Probe gleichzeitig auf der Platte, zusammen etwa 430 GB. Die Proben wurden nacheinander verarbeitet.

2.2 Haupt-Umgebung

Die Werkzeuge liegen in einer conda-Umgebung `na12878_pipeline`. Sie wurde so angelegt:

```
conda create -n na12878_pipeline \
  -c bioconda -c conda-forge python=3.10
conda activate na12878_pipeline
conda install bwa samtools bcftools gatk4 \
  fastqc multiqc fastp snakemake
conda install pigz stopit matplotlib
```

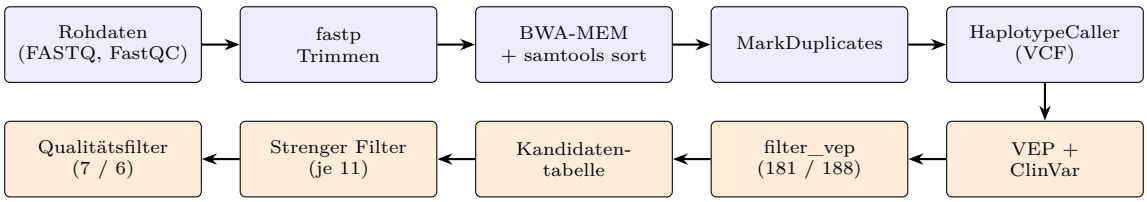


Abbildung 1: Ablauf der Pipeline. Blau: von den Reads zur Variantenliste. Orange: Annotation, Filterung und Prüfung. Zahlen: Mutter / Vater. Danach folgen Abbildungen und der Elternvergleich.

```
pip install pycirclize
```

Die Zusammensetzung lässt sich mit `conda env export -from-history` festhalten. `pycirclize` (und dessen Abhängigkeit Biopython) kommt über pip und ist daher nicht in dieser Liste enthalten.

2.3 VEP-Umgebung

VEP läuft in einer eigenen Umgebung `vep`, weil es als Paket für Intel-Prozessoren (`osx-64`) installiert wurde. Auf Apple Silicon läuft es dann über die Übersetzungsschicht Rosetta 2.

```
softwareupdate --install-rosetta \
  --agree-to-license
CONDA_SUBDIR=osx-64 conda create -n vep \
  -c conda-forge -c bioconda ensembl-vep
```

Der Workflow ruft VEP aus der Haupt-Umgebung mit `conda run -n vep -no-capture-output` auf, sodass keine Umgebung gewechselt werden muss.

2.4 VEP-Cache

VEP braucht lokal die Ensembl-Transkriptdaten, den sogenannten Cache. Der Installer `vep_install` brach bei der Übertragung mit einem FTP-Timeout ab. Stattdessen wurde das Archiv per HTTPS geladen (25,7 GB; `-C -` setzt einen abgebrochenen Download fort) und entpackt:

```
mkdir -p ~/.vep && cd ~/.vep
caffeinate -i curl -C - -O https://ftp.ensembl.org/pub/
      ↪ release-116/variation/indexed_vep_cache/
      ↪ homo_sapiens_vep_116_GRCh38.tar.gz
tar xzf homo_sapiens_vep_116_GRCh38.tar.gz
rm homo_sapiens_vep_116_GRCh38.tar.gz
```

`caffeinate -i` verhindert, dass der Mac während des Downloads in den Ruhezustand geht.

3 Projektstruktur

Alle Befehle werden im Projektordner `~/na12878_projekt` ausgeführt. Die Ordner entsprechen den Phasen der Pipeline:

```
na12878_projekt/
  Snakefile            Workflow
  *.py                 Auswertung, Filter, Abbildungen,
                       Elternvergleich
  rohdaten/            FASTQ (Eingabe)
  referenz/            Genom, Index, ClinVar
  qc/                  FastQC, MultiQC, flagstat
  trimmed/             getrimmte FASTQ
  alignment/           BAM-Dateien
  logs/                fastp- und Duplikat-Berichte
  varianten/           VCF- und TSV-Dateien
  abbildungen/<Probe>/ PNG und JSON je Probe
  abbildungen/11_...   Allelbalance beider Eltern
  .snakemake/          interne Snakemake-Daten
```

4 Daten und Referenz

4.1 Sequenzdaten

Verwendet werden zwei Läufe des Platinum-Genomes-Pedigrees (Eberle et al., 2017): ERR194147 (NA12878, im Pedigree die Mutter) mit 1.574.530.218 Reads und ERR194146 (NA12877, der Vater) mit 1.626.361.156 Reads, jeweils paired-end mit 101 bp. Die FASTQ-Dateien (`_1` und `_2`) liegen in `rohdaten/`. Die relative Tiefe der Geschlechtschromosomen bestätigt das Geschlecht: Mutter X 0,97 und Y 0,093, Vater X 0,50 und Y 0,411. Die Daten stammen aus dem European Nucleotide Archive (Illumina HiSeq 2000; ERR194147 gehört zum Projekt PRJEB3381, die Probe heißt dort `NA12878_30x`; die hier gemessene Tiefe beträgt etwa 50×). Die Projektzuordnung von ERR194146 habe ich nicht gesondert geprüft. Die Dateien liegen unter `ftp.sra.ebi.ac.uk/vol1/fastq/ERR194/`.

4.2 Referenzgenom

Als Referenz dient die Ensembl-Version GRCh38 der „primary assembly“ (Release 111). Sie enthält die Chromosomen und nicht platzierte Contigs, aber keine alternativen Haplotypen, die beim Mapping zu mehrdeutigen Positionen führen würden.

```
curl -O https://ftp.ensembl.org/pub/release-111/fasta/
      ↪ homo_sapiens/dna/Homo_sapiens.GRCh38.dna.
      ↪ primary_assembly.fa.gz
bwa index Homo_sapiens.GRCh38.dna.primary_assembly.fa.gz
samtools faidx Homo_sapiens.GRCh38.dna.primary_assembly.fa.
      ↪ gz
```

Dabei entstehen mehrere Hilfsdateien: der BWA-Index (`.bwt`, `.pac`, `.sa`, `.ann`, `.amb`), der samtools-Index (`.fai`, `.gzi`) und das Sequenzwörterbuch (`.dict`), das GATK verlangt. Es wurde mit `gatk CreateSequenceDictionary -R Referenz.fa.gz` erzeugt. Der BWA-Index basiert auf der Burrows-Wheeler-Transformation: Die Referenz wird so komprimiert und durchsuchbar gemacht, dass Reads in kurzer Zeit auf exakte Teilstücke abgeglichen werden können.

4.3 ClinVar

ClinVar (Landrum et al., 2018) wird als VCF mit Tabix-Index geladen. Die verwendete Datei trägt das Datum 2026-10-04.

```
cd referenz
curl -O https://ftp.ncbi.nlm.nih.gov/pub/clinvar/vcf_GRCh38/
      ↪ clinvar.vcf.gz
```

```
curl -O https://ftp.ncbi.nlm.nih.gov/pub/clinvar/vcf_GRCh38/
    ↪ clinvar.vcf.gz.tbi
```

ClinVar wird laufend aktualisiert. Wer den Lauf später wiederholt, erhält leicht abweichende Zahlen. Für exakte Reproduzierbarkeit sollte die verwendete Datei aufbewahrt werden.

5 Die Pipeline Schritt für Schritt

5.1 Wie Snakemake arbeitet

Snakemake (Mölder et al., 2021) beschreibt die Pipeline als Regeln mit Eingabe-, Ausgabedateien und einem Shell-Befehl. Aus den Dateinamen leitet es selbst die Reihenfolge ab: Eine Regel läuft, wenn ihre Ausgabe fehlt oder älter als ihre Eingabe ist. Dadurch wird nach einem Abbruch nur das Fehlende nachgeholt. Die Regel `all` legt das Ziel fest. Am Dateikopf stehen die Konstanten; Probenkennung und Name kommen aus der Konfiguration, Standard ist die Mutter:

```
SAMPLE = config.get("sample", "ERR194147")
NAME = config.get("name", "NA12878")
REF = "referenz/Homo_sapiens.GRCh38.dna.primary_assembly.fa.
    ↪ gz"
THREADS = 4
```

5.2 Qualitätskontrolle (FastQC)

FastQC zeigt pro Position die Basenqualität, den GC-Gehalt, Adapter- und Duplikatanteile der Rohdaten. So sieht man, ob die Sequenzierung brauchbar ist und ob getrimmt werden muss.

```
rule fastqc:
    input: r1 = f"rohdaten/{SAMPLE}_1.fastq.gz",
           r2 = f"rohdaten/{SAMPLE}_2.fastq.gz"
    output: html1 = f"qc/{SAMPLE}_1_fastqc.html",
            html2 = f"qc/{SAMPLE}_2_fastqc.html"
    threads: THREADS
    shell: "fastqc {input.r1} {input.r2} -o qc/ -t {threads}
      ↪ "
```

5.3 Trimmen (fastp)

Am Ende eines Fragments liest der Sequenzierer manchmal in den Adapter hinein, und Basen am Readende sind unsicherer. fastp (Chen et al., 2018) erkennt die Adapter bei Paired-End-Daten automatisch, kürzt sie und verwirft Reads schlechter Qualität. Es lief mit den Standardwerten (unter anderem Phred-Mindestqualität 15, höchstens 40 % schlechte Basen pro Read, höchstens 5 N, Mindestlänge 15 bp).

```
rule trimming:
    input: r1 = ..., r2 = ...
    output: r1 = f"trimmed/{SAMPLE}_1.trim.fastq.gz",
            r2 = f"trimmed/{SAMPLE}_2.trim.fastq.gz",
            json = f"logs/{SAMPLE}.fastp.json",
            html = f"logs/{SAMPLE}.fastp.html"
    shell: fastp -i {input.r1} -I {input.r2}
        -o {output.r1} -O {output.r2}
        -j {output.json} -h {output.html}
        -w {threads}
```

Ergebnis (Mutter): Von 1.574.530.218 Reads blieben 1.531.902.900 (97,3 %) übrig. Verworfen wurden 40.643.538 wegen niedriger Qualität und 1.983.780 wegen zu vieler N; bei 13.688.342 Reads wurden Adapter gekürzt. Der Q30-Anteil stieg von 92,08 auf 93,33 %. **Vater:** Von 1.626.361.156 Reads blieben 1.584.374.426 (97,4 %); 37.754.576 Reads wurden wegen niedriger Qualität und 4.232.154 wegen zu vieler N verworfen, bei 10.192.218 wurden Adapter gekürzt, der Q30-Anteil stieg von 90,65 auf 91,83 %.

5.4 Mapping (BWA-MEM)

Beim Mapping wird für jeden Read die Stelle im Genom gesucht, von der er stammt. BWA-MEM (Li, 2013) findet zuerst längste exakte Übereinstimmungen („maximal exact matches“) über den Burrows-Wheeler-Index und verlängert sie dann mit einem lokalen Alignment; bei Paaren nutzt es den Abstand und die Orientierung beider Reads. Die Ausgabe geht direkt in `samtools sort`, sodass keine riesige unsortierte SAM-Datei entsteht.

```
rule alignment:
    input: r1 = ..., r2 = ..., ref = REF
    output: bam = f"alignment/{SAMPLE}.sorted.bam"
    shell:
        bwa mem -t {threads}
          -R '@RG\tID:{SAMPLE}\tSM:{NAME}\tPL:ILLUMINA'
          {input.ref} {input.r1} {input.r2}
          | samtools sort -@ {threads} -o {output.bam} -
        samtools index {output.bam}
```

Die Option `-R` schreibt eine Read-Group (Probenname aus der Konfiguration, Plattform Illumina) in jede Zeile. GATK verlangt sie, damit klar ist, zu welcher Probe Reads gehören. **Ergebnis:** Mutter 99,74 % abgebildet, 97,93 % korrekt gepaart (Ausgabe etwa 96 GB); Vater 99,60 % und 97,66 %.

5.5 Duplikate markieren (GATK MarkDuplicates)

Aus einem DNA-Fragment können durch die PCR-Amplifikation mehrere identische Reads entstehen. Würde man sie mitzählen, sähe ein Fehler in einem Fragment wie viele unabhängige Belege aus. MarkDuplicates erkennt Reads mit gleichen Start- und Endpositionen und setzt ein Flag; die Reads bleiben in der Datei, der HaplotypeCaller überspringt sie aber.

```
rule mark_duplicates:
    input: bam = f"alignment/{SAMPLE}.sorted.bam"
    output: bam = f"alignment/{SAMPLE}.dedup.bam",
            metrics = f"logs/{SAMPLE}.dedup_metrics.txt"
    shell: gatk MarkDuplicates -I {input.bam}
        -O {output.bam} -M {output.metrics}
        samtools index {output.bam}
```

Ergebnis: Duplikationsrate Mutter 1,18 % (17.962.267 Reads; Ausgabe etwa 113 GB), Vater 1,19 % (18.730.877 Reads).

5.6 Variantenbestimmung (GATK HaplotypeCaller)

Der HaplotypeCaller (McKenna et al., 2010) sucht Regionen mit Abweichungen von der Referenz, setzt dort aus den Reads lokal neue Haplotypen zusammen und berechnet für jeden die Wahrscheinlichkeit. Daraus ergibt sich der Genotyp (0/0, 0/1 oder 1/1) mit

Qualitätswerten. Die Regel ruft ihn ohne zusätzliche Optionen auf (Standard-Ploidie 2, keine Zielregionen).

```
rule variant_calling:
    input: bam = f"alignment/{SAMPLE}.dedup.bam", ref = REF
    output: vcf = f"varianten/{SAMPLE}.vcf.gz"
    shell: gatk HaplotypeCaller -R {input.ref}
        -I {input.bam} -O {output.vcf}
```

Abweichend von den GATK Best Practices (Van der Auwera et al., 2013) gab es weder eine Rekalibrierung der Basenqualitäten (BQSR) noch eine Variantenfilterung (VQSR oder Hard-Filter). **Ergebnis:** 4.893.226 (Mutter) bzw. 4.889.622 (Vater) Varianten im ungefilterten VCF; Ti/Tv jeweils 1,98 (nur Chromosomen).

5.7 Annotation (VEP und ClinVar)

Eine Variante bekommt erst Bedeutung, wenn man weiß, was sie im Gen bewirkt. VEP (McLaren et al., 2016) ordnet jede Variante den Transkripten zu und benennt die Konsequenz mit Begriffen der Sequence Ontology (zum Beispiel `missense_variant`, `frameshift_variant`). Pro Variante entstehen mehrere Einträge, einer je Transkript; sie stehen im INFO-Feld `CSQ`. Mit `-custom` wird ClinVar als zusätzliche Spur angehängt.

```
rule vep_annotate:
    input: vcf = f"varianten/{SAMPLE}.vcf.gz",
           clinvar = "referenz/clinvar.vcf.gz"
    output: vcf = f"varianten/{SAMPLE}.vep.vcf.gz"
    shell: conda run -n vep --no-capture-output vep
        -i {input.vcf}
        --cache --offline --assembly GRCh38
        --dir_cache ~/.vep
        --custom file={input.clinvar},
          short_name=ClinVar,format=vcf,
          type=exact,coords=0,fields=CLNSIG
        --vcf --compress_output bgzip
        --fork {threads} -o {output.vcf}
```

`type=exact` verlangt, dass Position und Allel mit dem ClinVar-Eintrag übereinstimmen. Das Ergebnis steht als Feld `ClinVar_CLNSIG` in der `CSQ`-Zeile. Die ClinVar-Zuordnung geschieht also über Position und Allel, nicht über das Gen. **Ergebnis:** Mutter 4.864.560 annotierte Varianten, davon 44.476 mit ClinVar-Treffer; Vater 4.864.034 und 43.890. Etwa 0,6 % der gerufenen Varianten fehlen in beiden Ausgaben; die Ursache ist nicht vollständig geklärt (Contigs außerhalb des Caches sind eine plausible Teilerklärung). Die Referenz stammt aus Ensembl-Release 111, der Cache aus Release 116; die Genomsequenz ist in beiden gleich (GRCh38), nur die Genmodelle können sich unterscheiden.

5.8 Filtern auf pathogene Einträge

`filter_vep` behält Varianten, deren ClinVar-Wert zum Muster passt. Das Muster „athogenic“ ohne Anfangsbuchstaben trifft `Pathogenic` und `Likely_pathogenic`, aber auch `Conflicting_classifications_of_pathogenicity`.

```
rule filter_pathogenic:
    input: vcf = f"varianten/{SAMPLE}.vep.vcf.gz"
    output: vcf = f"varianten/{SAMPLE}.pathogene.vcf"
    shell: conda run -n vep --no-capture-output
        filter_vep -i {input.vcf}
        --filter "ClinVar_CLNSIG match athogenic"
        -o {output.vcf}
```

Ergebnis: Mutter 181 Varianten (170 widersprüchlich, 11 eindeutig), Vater 188 (177 und 11).

5.9 Kandidatentabelle, strenger Filter und Qualitätsklassen

Das VCF mit den verschachtelten `CSQ`-Einträgen ist schwer lesbar. Zwei kleine Python-Skripte machen daraus eine Tabelle. `auswertung.py` liest das `CSQ`-Format aus dem Kopf, schreibt pro Variante und Gen eine Zeile (Position, Allele, Gen, Konsequenz, ClinVar-Wert, Genotyp), ergänzt aus dem FORMAT-Feld Tiefe (DP), Genotypqualität (GQ), Read-Zahlen (AD) und die Allelbalance AB = Alt-Reads / (Ref- + Alt-Reads) und entfernt Dubletten. `kandidaten_filter.py` behält nur Zeilen mit `Pathogenic` oder `Likely_pathogenic`, einer proteinverändernden Konsequenz (Missense, Stop-Gain, Frameshift, Splice-Akzeptor und -Donor, Start-Verlust, Stop-Verlust, Inframe) und ohne den Hinweis auf Nonsense-vermittelten Abbau (NMD). Danach ordnet es jeder Zeile eine Klasse zu (Spalte `QUALITAET`):

- **belastbar:** heterozygot mit AB 0,35–0,75, DP $\geq$ 15 und GQ $\geq$ 30; homozygot (1/1) mit AB $\geq$ 0,85 und denselben Mindestwerten.
- **unsicher:** alle übrigen Fälle, auch Indels mit AB $<$ 0,25, weil der Caller den Alt-Anteil bei Indels systematisch unterschätzt.
- **Artefaktverdacht:** SNVs mit AB $<$ 0,25 (heterozygot) und homozygote Treffer mit AB $<$ 0,5. Sie werden nicht in die Ausgabe übernommen, sondern auf stderr gelistet.

Die Schwellen sind pragmatisch und nicht an einem Truth Set kalibriert.

```
rule kandidaten_tabelle:
    input: vcf = ".../pathogene.vcf",
           skript = "auswertung.py"
    output: tsv = ".../pathogene_tabelle.tsv"
    shell: "python {input.skript} {input.vcf} > {output.tsv}
      ↪ "

rule kandidaten_streng:
    input: tsv = ".../pathogene_tabelle.tsv",
           skript = "kandidaten_filter.py"
    output: tsv = ".../kandidaten_streng.tsv"
    shell: "python {input.skript} {input.tsv} > {output.tsv}
      ↪ "
```

Ergebnis: Der strenge ClinVar-Filter lieferte bei beiden Eltern 11 Treffer (Vater: 9 proteinverändernd). Nach dem Qualitätsfilter bleiben **7 Kandidaten bei der Mutter** und **6 beim Vater**, alle heterozygot.

5.10 Qualitätszusammenfassung

Eine weitere Regel sammelt die Zahlen aus fastp, MarkDuplicates und `samtools flagstat` in einer Textdatei (`qc/<Probe>.zusammenfassung.txt`). Sie gehört nicht zur Zielregel `all` und wird gezielt aufgerufen (siehe Abschnitt 9).

6 Technische Prüfung der Kandidaten

Ein ClinVar-Treffer sagt nichts darüber, ob die Variante im eigenen Datensatz echt ist. Deshalb wurden die Kandidaten zusätzlich geprüft:

- **Allelbalance (aus AD):** Eine echte heterozygote Variante liegt bei etwa 0,5; bei Indels ist der AD-Wert verzerrt.
- **Mapping-Qualität (MAPQ):** Mittelwert und Anteil der Reads mit MAPQ < 20 am Locus. Niedrige Werte weisen auf mehrdeutige Positionen hin, etwa bei Paralogen und Pseudogenen.
- **GATK-Hard-Filter-Schwellen (nur zur Beurteilung, nicht angewendet):** SNV mit QD < 2, QUAL < 30, SOR > 3, FS > 60, MQ < 40, MQRankSum $< -12{,}5$, ReadPosRankSum < -8; Indel mit QD < 2, QUAL < 30, FS > 200, ReadPosRankSum < -20.

Mutter (7 Kandidaten): belastbar sind *TPP1* (AB 0,66), *PAH* (0,61) und *PLG* (0,39). Unsicher sind *PRSS1* 7:142750561 (0,32; Region mit Paralogen), *SERPINA1* (0,17; FS 34,6, SOR 4,97), *VPS13B* (0,18; nur 3 Alt-Reads bei DP 17) und *PI4KA* (0,10; QD 1,12, 18,9 % der Reads mit niedriger MAPQ, Pseudogene). Als Artefaktverdacht verworfen wurden *TUBB8* (0,14), *CNN2* an zwei Positionen (0,20 und 0,17) und *PRSS1* 7:142750600 (0,12).

Vater (6 Kandidaten): belastbar sind *POLG* (0,53), *CCDC40* (0,62), *CFTR* (0,47; vier ClinVar-Sterne) und *CLCN1* (0,61). Unsicher sind *LAMB2* (0,28; QD 1,04, SOR 4,97, FS 44,9, null Sterne) und *SERPINA1* (0,13). Verworfen wurden *CNN2* an zwei Positionen (0,13) und *PRSS1* 7:142750561 (0,20).

Die Krankheitsangaben und Sterne stammen aus dem ClinVar-Feld `CLNDN` bzw. dem Prüfstatus; sie sind Einreicherangaben, keine Diagnosen. Dass *SERPINA1*, *PRSS1* und *CNN2* bei beiden Eltern an denselben Positionen mit niedriger Allelbalance auftauchen, deutet auf einen systematischen Mapping- oder Calling-Artefakt hin; die *SERPINA1*-Insertion ließ sich in den Roh-Alignments nicht bestätigen. Ein Abgleich mit Populationsfrequenzen (gnomAD) wurde nicht durchgeführt.

7 Elternvergleich

`eltern_vergleich.py` liest beide Dateien `kandidaten_streng.tsv` (nach Qualitätsfilter), schreibt sie gemeinsam nach `varianten/eltern_vergleich.tsv` und listet Gene, die bei beiden Eltern, nur bei der Mutter oder nur beim Vater vorkommen. Das Skript gehört nicht zu Snakemake.

```
python3 eltern_vergleich.py
python3 abbildung_paar.py   # Abb. 11
```

Ergebnis: Gemeinsam ist nur *SERPINA1* (dieselbe Variante, unsicher, nicht bestätigt). Nur bei der Mutter: *PAH*, *PI4KA*, *PLG*, *PRSS1*, *TPP1*, *VPS13B*. Nur beim Vater: *CCDC40*, *CFTR*, *CLCN1*, *LAMB2*, *POLG*. Es gibt also kein Gen mit belastbarer Variante bei beiden Eltern.

Die Deutung folgt der Mendelschen Logik: Tragen beide Eltern eine rezessive Variante im selben Gen, hat ein Kind ein Risiko von 25 % zu erkranken, 50 % sind Träger und 25 % tragen sie nicht. Trägt nur ein Elternteil die Variante, ist ein Kind mit 50 % Träger; bei dominant wirkenden Genen kann es bis zu 50 % die Variante erben. Der Erbgang steht nicht in den Tabellen und muss je Gen nachgeschlagen werden (*TPP1*, *PAH*, *CFTR*, *CCDC40* klassisch rezessiv, *POLG* und *CLCN1* je nach Krankheitsbild beides, *PRSS1* dominant). Das ist eine Demonstration auf öffentlichen Beispieldaten, keine Risikoabschätzung und keine Diagnose.

8 Visualisierung

Die Abbildungsskripte laufen pro Probe. Die Probe wird über die Umgebungsvariablen `SAMPLE` und `NAME` gewählt; Ausgabe und JSON-Zwischenwerte liegen in `abbildungen/<SAMPLE>/`. Beide Eltern erhalten dieselben Farben und Diagrammtypen. Mit `-neu` werden die Werte neu berechnet. Gestaltet wurde einheitlich mit einem Blauton und einem orangen Akzent, dünnen Elementen und direkter Beschriftung statt Legenden.

Die Sequenziertiefe wird aus `samtools idxstats` und der mittleren Leselänge geschätzt. Bei weiblichen Proben erwartet man für X etwa 1 und für Y etwa 0 relativ zum Autosomen-Median, bei männlichen jeweils etwa 0,5 (gemessen: Mutter 0,97 / 0,093; Vater 0,50 / 0,411).

9 Ausführung

Der Lauf beginnt mit einem Trockenlauf, der zeigt, welche Regeln ausgeführt würden. Die Mutter ist die Standardprobe; für den Vater wird die Probe über `-config` gesetzt, wobei das Ziel vor `-config` stehen muss:

```
cd ~/na12878_projekt
conda activate na12878_pipeline
snakemake -n
caffeinate -i snakemake --cores 4
# Vater (NA12877)
caffeinate -i snakemake --cores 4 \
  --config sample=ERR194146 name=NA12877
```

Qualitätszusammenfassung, Abbildungen und Elternvergleich sind nicht Teil von `all`. Pro Probe:

```
caffeinate -i snakemake --cores 4 \
  qc/ERR194146.zusammenfassung.txt \
  --config sample=ERR194146 name=NA12877
SAMPLE=ERR194146 NAME=NA12877 caffeinate -i bash -c \
 'python3 abbildungen.py > /dev/null && \
  python3 abbildungen2.py --neu && \
  python3 einzelgenom.py'
python3 abbildungen3.py
```

Tabelle 3: Abbildungen pro Probe und zugehörige Skripte (Nr. 11 gilt für beide Eltern).

Nr.	Datei	Skript	Inhalt
1	`1_trichter.png`	`abbildungen.py`	Filtertrichter bis zu den Kandidaten
2	`2_konsequenzen.png`	`abbildungen.py`	VEP-Konsequenzen
3	`3_clinvar.png`	`abbildungen.py`	ClinVar-Klassen
4	`4_chromosomen.png`	`abbildungen.py`	Varianten pro Chromosom
5	`5_dichte_kandidaten.png`	`abbildungen2.py`	Heatmap der Variantendichte mit Kandidaten
6	`6_kandidaten_qualitaet.png`	`abbildungen2.py`	Kandidaten mit Krankheit und Qualität
7	`7_circos.png`	`einzelgenom.py`	Zirkuläre Genomkarte (Circos)
8	`8_variantenstatistik.png`	`einzelgenom.py`	Typen, Ti/Tv, Indel-Längen, Heterozygotie
9	`9_coverage_chromosomen.png`	`einzelgenom.py`	Sequenziertiefe pro Chromosom
10	`10_sequenzierung_mapping.png`	`abbildungen3.py`	Reads pro Schritt, entfernt und markiert
11	`11_allelbalance_eltern.png`	`abbildung_paar.py`	Allelbalance der Treffer beider Eltern

Tabelle 4: Laufzeiten der Schritte laut Snakemake-Protokoll (Mutter).

Schritt	Dauer
fastp (Trimmen)	22 min
BWA-MEM und Sortieren	48 h
MarkDuplicates	19 h
HaplotypeCaller	40 h
VEP-Annotation	45 min
`filter_vep`	21 min
Qualitätszusammenfassung	1,5 min

Die Reihenfolge ist wichtig: `einzelgenom.py` liest `kandidaten_werte.json`, das `abbildungen2.py` erzeugt; `abbildungen3.py` braucht `qc/` und `logs/` der Probe. Danach folgen `eltern_vergleich.py` und `abbildung_paar.py`. `-cores` ist bei Snakemake 6 Pflicht (`-c` gibt es nicht). Ändert man ein Skript, das Eingabe einer Regel ist (etwa `kandidaten_filter.py`), läuft nur diese Regel erneut.

Die Laufzeiten in Tabelle 4 stammen aus den Snakemake-Protokollen der Mutter; die vier großen Schritte dauerten zusammen etwa 108 Stunden (4,5 Tage). Die gesamte Pipeline des Vaters dauerte rund 45,5 Stunden; die Zeiten hängen stark von der Auslastung des Laptops ab. GATK arbeitet die Chromosomen in der alphabetischen Reihenfolge der Referenz ab, daher entspricht die Position im Fortschrittsprotokoll des HaplotypeCallers nicht dem linearen Fortschritt.

10 Fehlersuche

Folgende Probleme traten tatsächlich auf:

`perl: Bad CPU type` VEP ist ein Intel-Programm. Lösung: Rosetta 2 installieren (`softwareupdate -install-rosetta`).

Timeout bei `vep_install` Der FTP-Download brach ab. Lösung: Cache-Archiv per `curl -C -` über HTTPS laden und entpacken.

`zcat` auf macOS Das BSD-`zcat` liest `.gz`-Dateien nicht. Lösung: `gzip -dc datei.gz`.

`no matches found` in zsh Ein Platzhalter ohne Treffer bricht den ganzen Befehl ab. Lösung: Ordner explizit angeben statt `fastp*`.

Schleifenvariable wird nicht getrennt In zsh zerlegt `set - $p` den Inhalt nicht. Lösung: `c=${p% *}; pos=${p#* }`.

Eingefügter Code bricht Python-Code in einer Zeile eingefügt, verliert die Einrückung. Lösung: Dateien mit `cat > datei « 'EOF'` anlegen.

`-c4` bei Snakemake Snakemake 6 verlangt `-cores`.

`ModuleNotFoundError: matplotlib` Fehlendes Paket in der Umgebung. Lösung: `conda install -n na12878_pipeline -c conda-forge matplotlib`.

`sed -i` auf macOS Das BSD-`sed` verlangt ein Argument: `sed -i " ...`.

Scheintreffer unter den pathogenen Varianten Ohne Variantenfilter erschienen Treffer mit Allelbalance 0,13–0,32 (etwa *CNN2*, *PRSS1*). Lösung: Qualitätsklassen in `kandidaten_filter.py`; die Kandidatenliste der Mutter sank dadurch von 11 auf 7.

Indels fälschlich verworfen Die erste Fassung wandte AB $< 0{,}25$ auch auf Indels an. Lösung: Indels gelten dann nur als „unsicher“.

Falscher Ausgabepfad nach Skript-Patch Ein automatisches Ersetzen machte aus `OUT` einen verschachtelten Pfad. Lösung: Zeile mit `grep -n` prüfen.

Ziel und `-config` Das Snakemake-Ziel muss vor `-config` stehen.

11 Grenzen und Hinweise

- **Keine Validierung gegen ein Truth Set.** Sensitivität und Präzision der Variantenbestimmung sind nicht quantifiziert. Vorgesehen ist ein Vergleich mit Platinum-Genomes- oder GIAB-Referenzcalls (Eberle et al., 2017; Zook et al.,

2014), z. B. mit rtg vcfeval oder hap.py; die Chromosomennamen (`1` gegenüber `chr1`) müssen angeglichen werden.

- Das VCF ist ungefiltert (kein VQSR, Hard Filter oder BQSR). Die Qualitätsklassen gelten nur für die ClinVar-Kandidaten, ihre Schwellen sind nicht kalibriert.
- Treffer in Regionen mit Paralogen, Pseudogenen oder hohem GC-Gehalt sind anfällig für Mapping-Artefakte; die *SERPINA1*-Insertion wurde in den Roh-Alignments nicht bestätigt.
- Auf dem Y-Chromosom stehen in beiden Proben rund 14.000 Varianten, überwiegend Mapping-Artefakte.
- Etwa 0,6 % der gerufenen Varianten fehlen in der VEP-Ausgabe; die Ursache ist nicht vollständig geklärt.
- Referenz (Release 111) und VEP-Cache (Release 116) stammen aus verschiedenen Ensembl-Versionen.
- ClinVar ändert sich laufend (hier Stand 2026-10-04). Sterne und Krankheitsnamen sind wörtlich übernommen, nicht fachlich neu bewertet.
- Abbildungs- und Vergleichsskripte sind nicht in Snakemake eingebunden.
- `alignment/<Probe>.sorted.bam` wird nach der Duplikatmarkierung nicht mehr gebraucht.
- Alle Ergebnisse sind ClinVar-Annotationen auf öffentlichen Beispieldaten, keine Diagnosen.

A Skripte

auswertung.py

```
import sys
from collections import Counter
fields = None
seen = set()
zaehler = Counter()
print("CHROM\tPOS\tREF\tALT\tGEN\tKONSEQUENZ\tCLINVAR\
    ↪ tGENOTYP\tAD\tDP\tGQ\tAB")
for line in open(sys.argv[1]):
    if line.startswith("##INFO=<ID=CSQ"):
        fields = line.split("Format: ")[1].rstrip('">\n').
    ↪ split("|")
    elif line.startswith("#"):
        continue
    else:
        c = line.rstrip("\n").split("\t")
        info = dict(x.split("=", 1) for x in c[7].split(";")
    ↪ if "=" in x)
        gt, ad, dp, gq, ab = "", ".", ".", ".", "NA"
        if len(c) > 9:
            fmt = dict(zip(c[8].split(":"), c[9].split(":")))
    ↪
            gt = fmt.get("GT", "")
            ad = fmt.get("AD", ".")
            dp = fmt.get("DP", ".")
            gq = fmt.get("GQ", ".")
            try:
                n = [int(x) for x in ad.split(",")]
                if sum(n) > 0:
                    ab = f"{sum(n[1:]) / sum(n):.2f}"
            except ValueError:
                pass
        for e in info.get("CSQ", "").split(","):
            d = dict(zip(fields, e.split("|")))
            cs = d.get("ClinVar_CLNSIG", "")
            if "athogenic" in cs:
                key = (c[0], c[1], c[3], c[4], d.get("SYMBOL
    ↪ ", ""), d.get("Consequence", ""), cs, gt, ad, dp, gq
    ↪ , ab)
                if key not in seen:
                    seen.add(key)
                    zaehler[cs] += 1
                    print("\t".join(key))
print("\nZusammenfassung:", file=sys.stderr)
for k, v in zaehler.most_common():
    print(f"  {v:4d}  {k}", file=sys.stderr)
```

kandidaten_filter.py

```
import re
import sys

STRENG = re.compile(r"^(Pathogenic|Likely_pathogenic)")
PROTEIN = re.compile(r"missense|stop_gained|frameshift|
    ↪ splice_acceptor|splice_donor|start_lost|stop_lost|
    ↪ inframe")

def qualitaet(gt, ab, dp, gq, ref="", alt=""):
    try:
        ab, dp, gq = float(ab), int(dp), int(gq)
    except ValueError:
        return "unsicher"
    if gt in ("1/1", "1|1"):
        if ab < 0.5:
            return "artefaktverdacht"
        return "belastbar" if (ab >= 0.85 and dp >= 15 and gq
    ↪  >= 30) else "unsicher"
    if ab < 0.25:
        indel = any(len(a) != len(ref) for a in alt.split(","
    ↪ ))
        return "unsicher" if indel else "artefaktverdacht"
    if 0.35 <= ab <= 0.75 and dp >= 15 and gq >= 30:
        return "belastbar"
    return "unsicher"

gesehen = set()
verworfen = []
with open(sys.argv[1]) as f:
    print(f.readline().rstrip("\n") + "\tQUALITAET")
    for line in f:
        c = line.rstrip("\n").split("\t")
        chrom, pos, ref, alt, gen, kons, clin, gt, ad, dp,
    ↪ gq, ab = c[:12]
        if STRENG.match(clin) and PROTEIN.search(kons) and "
    ↪ NMD" not in kons:
            schluessel = (chrom, pos, ref, alt, gen)
            if schluessel in gesehen:
                continue
            gesehen.add(schluessel)
            q = qualitaet(gt, ab, dp, gq, ref, alt)
            if q == "artefaktverdacht":
                verworfen.append(f"  {gen:9s} {chrom}:{pos}
    ↪ AB={ab} DP={dp} GQ={gq}")
            else:
                print("\t".join(c[:12]) + "\t" + q)
print(f"\nVerworfen (Artefaktverdacht, AB < 0.25): {len(
    ↪ verworfen)}", file=sys.stderr)
for v in verworfen:
    print(v, file=sys.stderr)
```

qc_zusammenfassung.py

```
import json, sys

fastp_json, dedup_txt, flagstat_txt = sys.argv[1:4]

d = json.load(open(fastp_json))
s = d["summary"]
print("== fastp (Rohdaten -> getrimmt) ==")
for k, name in (("before_filtering", "vorher"), ("
    ↪ after_filtering", "nachher")):
    x = s[k]
    print(f"{name}: Reads {x['total_reads']:,}  Basen {x['
    ↪ total_bases']:,}  "
          f"Q30 {x['q30_rate'] * 100:.2f} %  GC {x['
    ↪ gc_content'] * 100:.1f} %  "
```

```
        f"Leselänge Ø {x['read1_mean_length']} bp")
print("Filterergebnis:", d.get("filtering_result"))
print("Adapter-Trimming:", d.get("adapter_cutting"))
print("Duplikationsrate (fastp-Schätzung):", d.get("
    ↪ duplication"))

print("\n== Duplikate (GATK MarkDuplicates) ==")
zeilen = open(dedup_txt).read().splitlines()
i = next(k for k, l in enumerate(zeilen) if l.startswith("##
    ↪ METRICS CLASS"))
m = dict(zip(zeilen[i + 1].split("\t"), zeilen[i + 2].split(
    ↪ "\t")))
for k in ("READ_PAIRS_EXAMINED", "UNPAIRED_READS_EXAMINED",
    ↪ "READ_PAIR_DUPLICATES",
          "PERCENT_DUPLICATION", "ESTIMATED_LIBRARY_SIZE"):
    print(f"{k}: {m.get(k)}")

print("\n== Mapping (samtools flagstat, dedup.bam) ==")
print("".join(open(flagstat_txt).readlines()[:12]))
```

eltern_vergleich.py

```
import csv

def lade(pfad):
    with open(pfad, encoding="utf-8") as f:
        return list(csv.DictReader(f, delimiter="\t"))

mutter = lade("varianten/ERR194147.kandidaten_streng.tsv")
vater = lade("varianten/ERR194146.kandidaten_streng.tsv")

def schluessel(z):
    return (z["CHROM"], z["POS"], z["REF"], z["ALT"])

def nach_gen(zeilen):
    d = {}
    for z in zeilen:
        d.setdefault(z["GEN"], []).append(z)
    return d

gm, gv = nach_gen(mutter), nach_gen(vater)
gemeinsam = sorted(set(gm) & set(gv))
nur_m = sorted(set(gm) - set(gv))
nur_v = sorted(set(gv) - set(gm))

ausgabe = []
def zeile(kat, gen, z, wer):
    hinweis = "X/Y: haploid beim Vater, Genotyp prüfen" if z
    ↪ ["CHROM"] in ("X", "Y") else ""
    ausgabe.append([kat, gen, wer, z["CHROM"], z["POS"], z["
    ↪ REF"], z["ALT"],
                    z["GENOTYP"], z["KONSEQUENZ"], z["
    ↪ CLINVAR"], hinweis])

print(f"Kandidaten: Mutter {len(mutter)}, Vater {len(vater)}
    ↪ ")
print(f"\nGene bei BEIDEN Eltern ({len(gemeinsam)}): mö
    ↪ gliches 25 %-Risiko, falls rezessiv")
for g in gemeinsam:
    km = {schluessel(z) for z in gm[g]}
    kv = {schluessel(z) for z in gv[g]}
    art = "gleiche Variante" if km & kv else "verschiedene
    ↪ Varianten (compound-heterozygot möglich)"
    print(f"  {g}: {art}")
    for z in gm[g]: zeile("beide:" + art, g, z, "Mutter")
    for z in gv[g]: zeile("beide:" + art, g, z, "Vater")

print(f"\nNur Mutter ({len(nur_m)}): {', '.join(nur_m)}")
for g in nur_m:
    for z in gm[g]: zeile("nur_Mutter", g, z, "Mutter")
print(f"\nNur Vater ({len(nur_v)}): {', '.join(nur_v)}")
for g in nur_v:
    for z in gv[g]: zeile("nur_Vater", g, z, "Vater")

with open("varianten/eltern_vergleich.tsv", "w", encoding="
    ↪ utf-8", newline="") as f:
    w = csv.writer(f, delimiter="\t")
    w.writerow(["KATEGORIE", "GEN", "ELTERNTEIL", "CHROM", "
    ↪ POS", "REF", "ALT",
                "GENOTYP", "KONSEQUENZ", "CLINVAR", "HINWEIS
    ↪ "])
    w.writerows(ausgabe)
print("\nGeschrieben: varianten/eltern_vergleich.tsv")
```

Literatur

Shifu Chen, Yanqing Zhou, Yaru Chen, and Jia Gu. fastp: an ultra-fast all-in-one FASTQ preprocessor. *Bioinformatics*, 34(17):i884–i890, 2018. doi: 10.1093/bioinformatics/bty560.

Michael A. Eberle, Epameinondas Fritzilas, Peter Krusche, et al. A reference data set of 5.4 million phased human variants validated by genetic inheritance from sequencing a three-generation 17-member pedigree. *Genome Research*, 27(1):157–164, 2017. doi: 10.1101/gr.210500.116.

Melissa J. Landrum, Jennifer M. Lee, Mark Benson, et al. ClinVar: improving access to variant interpretations and supporting evidence. *Nucleic Acids Research*, 46(D1): D1062–D1067, 2018. doi: 10.1093/nar/gkx1153.

Heng Li. Aligning sequence reads, clone sequences and assembly contigs with BWA-MEM. arXiv:1303.3997, 2013. URL `https://arxiv.org/abs/1303.3997`.

Aaron McKenna, Matthew Hanna, Eric Banks, et al. The Genome Analysis Toolkit: a MapReduce framework for analyzing next-generation DNA sequencing data. *Genome Research*, 20(9):1297–1303, 2010. doi: 10.1101/gr.107524.110.

William McLaren, Laurent Gil, Sarah E. Hunt, et al. The Ensembl Variant Effect Predictor. *Genome Biology*, 17: 122, 2016. doi: 10.1186/s13059-016-0974-4.

Felix Mölder, Kim Philipp Jablonski, Brice Letcher, et al. Sustainable data analysis with Snakemake. *F1000Research*, 10:33, 2021. doi: 10.12688/f1000research.29032.2.

Geraldine A. Van der Auwera, Mauricio O. Carneiro, Chris Hartl, et al. From FastQ data to high-confidence variant calls: the Genome Analysis Toolkit best practices pipeline. *Current Protocols in Bioinformatics*, 43(1):11.10.1–11.10.33, 2013. doi: 10.1002/0471250953.bi1110s43.

Justin M. Zook, Brad Chapman, Jason Wang, et al. Integrating human sequence data sets provides a resource of benchmark SNP and indel genotype calls. *Nature Biotechnology*, 32(3):246–251, 2014. doi: 10.1038/nbt.2835.